FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3133 – HIGH PERFORMANCE DATA PROCESSING

SECTION 2

PROJECT 1

OPTIMIZING HIGH-PERFORMANCE DATA PROCESSING FOR LARGE-SCALE WEB CRAWLERS

LECTURER: DR. SEAH CHOON SEN

GROUP ABC

STUDENT NAME	MATRIC NO
JOANNE CHING YIN XUAN	A23CS0227
LIM YU HAN	A23CS0241
CHUA JIA LIN	A23CS0069
EVELYN GOH YUAN QI	A23CS0222

TABLE OF CONTENTS

1.0 INTRODUCTION 2
1.1 PROJECT BACKGROUND 3
1.2 OBJECTIVES 3
1.3 TARGET WEBSITE AND DATA TO BE EXTRACTED 4
2.0 SYSTEM DESIGN & ARCHITECTURE 5
2.1 SYSTEM ARCHITECTURE 5
2.2 TOOLS AND FRAMEWORKS USED 6
2.3 ROLES OF TEAM MEMBERS 7
3.0 DATA COLLECTION 8
3.1 CRAWLING METHOD 8
3.2 NUMBER OF RECORDS COLLECTED 9
3.3 ETHICAL CONSIDERATIONS 10
4.0 DATA PROCESSING 10
4.1 DATA CLEANING METHODS 11
4.1.1 WHITESPACE NORMALIZATION 11
4.1.2 MISSING VALUE STANDARDIZATION 11
4.1.3 DUPLICATE RECORD REMOVAL 11
4.1.4 RECORD VALIDATION 12
4.1.5 DERIVED FEATURE GENERATION 12
4.2 DATA STRUCTURE 12
4.3 TRANSFORMATION & FORMATTING 14
4.3.1 TEXT NORMALIZATION 14
4.3.2 DATA TYPE ENFORCEMENT 14
4.3.3 DEDUPLICATION 15
4.3.4 DATA SERIALIZATION 15
4.3.5 ANALYTICAL TRANSFORMATION USING POLARS 15
5.0 OPTIMIZATION TECHNIQUES 16
5.1 METHOD 1 – MULTITHREAED WEB CRAWLING 16
5.2 METHOD 2 – PARALLEL DATA CLEANING 17

5.3 METHOD 3 – POLARS LAZY EXECUTION 18
6.0 PERFORMANCE EVALUATIONS 20
6.1 BEFORE VS AFTER OPTIMIZATION 20
6.2 COMPARISON OF TIME, MEMORY, CPU USAGE, THROUGHPUT 21
6.2.1 WEB CRAWLING PERFORMANCE COMPARISON 21
6.2.2 DATA PROCESSING PERFORMANCE COMPARISON 24
7.0 CHALLENGES & LIMITATIONS 27
7.1 CHALLENGES ENCOUNTERED 27
7.2 LIMITATIONS 27
8.0 CONCLUSION & FUTURE WORK 28
8.1 SUMMARY OF FINDINGS 28
8.2 FUTURE IMPROVEMENT 29
9.0 REFERENCES 30
10.0 APPENDICES 30

1.0 INTRODUCTION

This section introduces the background, objectives, and target websites of the project. It explains the importance of web crawling, the motivation for collecting large-scale news article metadata, and provides an overview of the data extracted from the New Straits Times (NST) website.

1.1 PROJECT BACKGROUND

Nowadays, news websites publish a large amount of information every day. Websites such as the **New Straits Times (NST)** regularly update their pages with the latest news on politics, business, sports, education, technology, and many other topics. This information is useful for research, data analysis, and machine learning. However, collecting thousands of news articles manually is very time-consuming and difficult. Therefore, web crawling is a practical and efficient solution for collecting large amounts of data automatically.

Web crawling is the process of using a computer program to visit web pages and extract the required information automatically. Compared to manual data collection, web crawling is much faster, more efficient, and can collect a large number of records with consistent results. The collected data can then be stored in a structured format for further processing and analysis.

In this project, a high-performance web crawler is developed to collect more than **100,000** article metadata records from the New Straits Times (NST) website. The crawler uses the NST XML sitemap to discover article URLs before extracting information such as the headline, publication date, author, section, summary, keywords, and body preview. To improve system performance, three optimization techniques are implemented. **Multithreading** is used for faster web crawling, **multiprocessing** is used for parallel data cleaning, and **Polars Lazy Execution** is used for efficient analytical processing. Finally, the performance of each technique is evaluated by comparing execution time, CPU usage, memory usage, and throughput.

1.2 OBJECTIVES

The objectives of this project are:

- To develop an automated web crawler that extracts article metadata from the New Straits Times (NST) website.
- To collect a large-scale structured dataset of news articles and store the extracted information in JSONL and CSV formats.
- To implement optimization techniques, including multithreading, multiprocessing, and Polars Lazy Execution, to improve crawling and data processing performance.
- To clean and transform the collected data by removing duplicate records, standardizing text fields, and generating additional analytical features.
- To evaluate the performance of the developed crawler by comparing execution time, CPU usage, memory usage, and throughput before and after optimization.

1.3 TARGET WEBSITE AND DATA TO BE EXTRACTED

The target website for this project is the **New Straits Times (NST)**, one of Malaysia's leading English-language news websites. The website was selected because it publishes a wide variety of news articles and provides an XML sitemap that allows article URLs to be discovered efficiently.

The crawler extracts article metadata instead of the full article content. Table 1.1 shows the collected information and a brief description of the attributes. The attributes extracted including the record_id, url, headline, publication_date, modified_date, section, author, summary, keywords, body_preview, word_count, and collected_at. The raw data are first stored in JSONL format before being cleaned and exported as a CSV file for further analysis.

Table 1.1: Data Extracted from NST Website

Attributes	Description
record_id	Unique identifier of the article
url	Link to the news article
headline	Title of the article

publication_date	Date the article was published
modified_date	Last updated date
section	News category
author	Name of the author
summary	Short description of the article
keywords	Article keywords
body_preview	Preview of the article content
word_count	Number of the words in the preview
collected_at	Date and time the data were collected

2.0 SYSTEM DESIGN & ARCHITECTURE

This section describes the overall design and architecture of the proposed NST web crawler. It explains the system architecture, the software tools and frameworks used, and the responsibilities of each team member throughout the project development.

2.1 SYSTEM ARCHITECTURE

Figure 2.1 demonstrates the system architecture of the proposed crawler pipeline. The proposed pipeline consists of six main stages, including sitemap discovery, web crawling, data storage, data processing, performance benchmarking, and visualization. The crawler first reads the NST XML sitemap to discover article URLs before downloading individual news pages. The extracted metadata is stored in JSONL format and exported as CSV files.

After data collection, the raw dataset undergoes data cleaning and transformation to remove duplicate records, standardize text fields, and generate additional features. The cleaned dataset is then processed using three different approaches, which are baseline Pandas processing, multiprocessing, and Polars Lazy Execution. Finally, benchmark results are recorded and visualized to compare execution time, throughput, CPU usage, and memory usage.

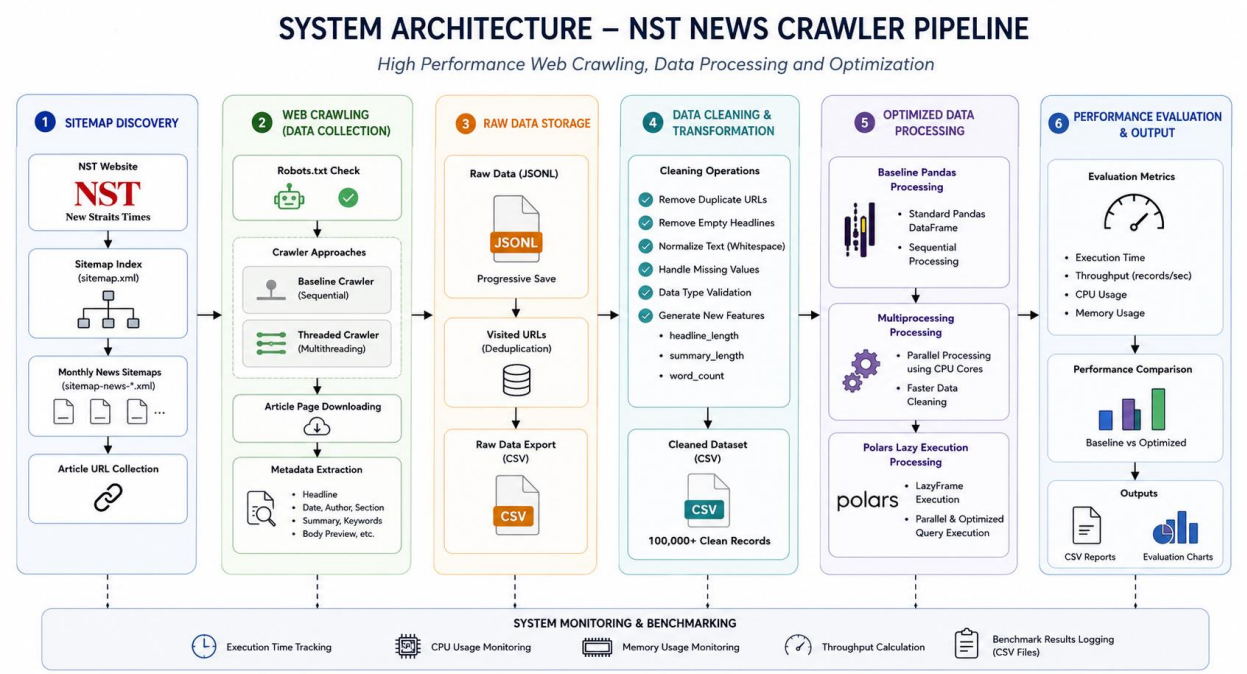

Figure 2.1: System Architecture for NST News Article Crawler Pipeline

2.2 TOOLS AND FRAMEWORKS USED

Several tools and frameworks were used to develop the proposed system. Table 2.1 shows the tools and frameworks used in this project. **Python** was selected as the main programming language because of its extensive support for web crawling and data processing libraries. Besides, the **Requests** library was used to retrieve web pages, while **BeautifulSoup** was used to parse HTML content and extract article metadata. Moreover, the **ThreadPoolExecutor** was implemented to perform multithreaded crawling, improving the efficiency of downloading multiple web pages simultaneously.

For data processing, **Pandas** was used for baseline data cleaning, **multiprocessing** enabled parallel processing across multiple CPU cores, and **Polars** with LazyFrame execution was used to optimize analytical processing on the cleaned dataset. In addition, **psutil** recorded the CPU usage, memory usage, and throughput during benchmarking, while **Matplotlib** generated comparison charts for performance evaluation.

Table 2.1: Tools and Frameworks Used

Tool / Framework	Purpose
Python	Main programming language
Requests	Send HTTP requests to retrieve web pages
BeautifulSoup	Parse HTML and extract article metadata
ThreadPoolExecutor	Perform multithreaded web crawling
Pandas	Baseline data cleaning and processing
Multiprocessing	Parallel data cleaning
Polars	High-performance analytical data processing
psutil	Monitor CPU, memory, and throughput
Matplotlib	Generate performance comparison charts

2.3 ROLES OF TEAM MEMBERS

The project was completed by dividing the development tasks among the team members. Each member was responsible for a different stage of the project to ensure that the crawler, optimization techniques, benchmarking, and documentation were completed efficiently. Table 2.2 shows the role and responsibilities of each team member.

Table 2.2: Roles and Responsibilities of Each Team Member

Team Member	Role	Responsibilities
Chua Jia Lin	System Architect	Designed the overall system architecture, integrated all project components, and reviewed the final deliverables.
Joanne Ching Yin Xuan	Web Crawling Engineer	Developed the baseline and multithreaded web crawler, implemented sitemap discovery, metadata extraction, URL management, and raw data collection from the NST website.

Lim Yu Han	Data Processing & Optimization Engineer	Developed the data cleaning pipeline, implemented multiprocessing and Polars Lazy Execution, optimized data processing performance, and generated the cleaned dataset.
Evelyn Goh Yuan Qi	Performance Evaluation Engineer	Conducted benchmark experiments, analyzed execution time, CPU usage, memory usage, and throughput, generated performance charts, and prepared the technical report and project documentation.

3.0 DATA COLLECTION

This section explains the process of collecting news article metadata from the New Straits Times (NST) website. It describes the crawling method, summarizes the number of records collected, and discusses the ethical considerations followed during the data collection process to ensure responsible web crawling.

3.1 CRAWLING METHOD

Figure 3.1 illustrates the workflow of the NST News Crawler. The crawler starts by accessing the **NST XML sitemap**, which contains links to news articles published on the NST website. Instead of searching every webpage one by one, the crawler reads the sitemap to collect article URLs more quickly and efficiently. This method reduces unnecessary requests and helps the crawler to find valid news articles.

Before downloading each article, the crawler checks the website's **robots.txt** file to make sure that crawling is allowed. Besides, the crawler also uses a custom **User-Agent** to identify itself and waits for a short period between requests. This delay helps to reduce the load on the website and allows the crawler to collect data responsibly.

Once an article page is downloaded using the **Requests** library, the crawler uses **BeautifulSoup** to extract important metadata, including the article headline, publication date, modified date, author, section, summary, keywords, and body preview. Each extracted record is saved in a

JSONL file, and the article URL is stored in a separate visited list to prevent the same article from being crawled again. After all articles have been collected, the raw data are exported to a **CSV** file for data cleaning and further analysis.

To measure the effectiveness of optimization, the project implements two crawling methods. The first method is a **baseline crawler**, which downloads articles one at a time. The second method is a **threaded crawler**, which uses multiple threads to download several articles simultaneously. Performance metrics such as execution time, throughput, CPU usage, and memory usage are recorded for comparison during benchmarking.

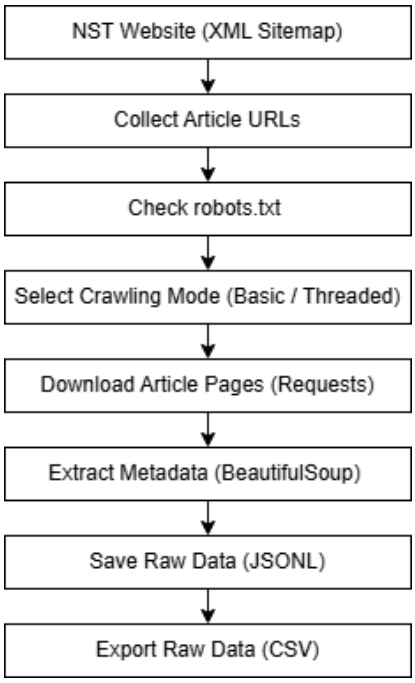

Figure 3.1: Workflow of NST News Crawler

3.2 NUMBER OF RECORDS COLLECTED

The objective of this project is to collect more than **100,000** news article metadata records from the New Straits Times (NST) website. During the crawling process, each extracted article is stored as a raw record in **JSONL** format. After the crawling process is completed, the raw dataset is exported to a **CSV** file to make it easier for viewing and further processing. At this stage, the

dataset contains the original metadata extracted from the website without any cleaning or transformation.

The developed crawler successfully collected **102,265** raw article metadata records, exceeding the target of 100,000 records. The collected dataset includes information such as the article URL, headline, publication date, modified date, section, author, summary, keywords, body preview, word count, and collection timestamp.

3.3 ETHICAL CONSIDERATIONS

Ethical web crawling is important to ensure that data collection does not negatively affect the target website. In this project, a few measures were taken to perform responsible crawling.

First, the crawler checks the **robots.txt** file before accessing the sitemap and article pages. Only URLs that are permitted by the website's crawling policy are processed.

Besides, a custom **User-Agent** is included in every HTTP request to identify the crawler. A configurable request delay is also applied to avoid sending too many requests in a short period, reducing the workload on the NST server.

Moreover, the crawler collects only **publicly available article metadata** and does not attempt to bypass website security, authentication, or access restrictions.

Furthermore, the collected data is used completely for academic purposes, including performance benchmarking and data processing.

These practices help ensure that the project complies with responsible web crawling principles while respecting the website's policies and resources.

4.0 DATA PROCESSING

Data processing is a critical phase of the proposed web crawling pipeline that transforms the raw data collected from the New Straits Times (NST) website into a structured, consistent, and analysis-ready dataset. This phase aims to improve data quality by eliminating inconsistencies, standardizing data formats, and generating additional analytical features. The data processing

workflow consists of three main stages: data cleaning, data structuring, and data transformation, ensuring that the processed dataset is accurate, complete, and suitable for subsequent analysis and reporting.

4.1 DATA CLEANING METHODS

Data cleaning is the first stage of the data processing pipeline, transforming the raw data collected by the web crawler into a structured, consistent, and reliable dataset suitable for subsequent analysis. The cleaning process was implemented in the `clean_nst.py` module and comprises a series of sequential operations designed to improve data quality by removing inconsistencies, handling incomplete records, and generating additional analytical attributes. These operations ensure that the processed dataset is accurate, complete, and suitable for further analytical processing.

4.1.1 WHITESPACE NORMALIZATION

The first cleaning step involves normalizing textual data by replacing multiple whitespace characters, including spaces, tabs, and newline characters, with a single whitespace character. This process standardizes the formatting of textual fields such as the article headline, summary, and body preview, thereby improving readability and ensuring consistency throughout the dataset.

4.1.2 MISSING VALUE STANDARDIZATION

After text normalization, missing or null values are standardized by replacing them with empty strings. This approach prevents data type inconsistencies during processing and minimizes the occurrence of runtime errors when exporting the cleaned dataset into CSV format.

4.1.3 DUPLICATE RECORD REMOVAL

Duplicate records are identified and removed using the article's URL as the unique identifier. Since each news article published by New Straits Times has a unique URL, duplicate entries generated during the crawling process can be accurately detected and eliminated. This process ensures that

each article appears only once in the final dataset, thereby improving data integrity and preventing biased analytical results.

4.1.4 RECORD VALIDATION

To ensure the quality and completeness of the dataset, records with missing or empty headline fields are excluded from the cleaned dataset. The headline serves as the primary identifier of a news article and is considered essential metadata for indexing, categorization, and subsequent analysis.

4.1.5 DERIVED FEATURE GENERATION

Following the cleaning process, several additional analytical attributes are generated to enrich the dataset. These derived features provide quantitative information regarding the textual content of each article and support descriptive and statistical analysis.

The generated features include:

- **Headline Length** – Calculates the total number of characters contained in the normalized headline.
- **Summary Length** – Calculates the total number of characters contained in the normalized summary.
- **Word Count** – Calculates the total number of words contained in the combined headline, summary, and body preview.

The inclusion of these derived attributes enhances the analytical value of the dataset by providing measurable indicators that can be utilized for exploratory data analysis and future machine learning applications.

Overall, the implemented data cleaning pipeline produces a complete, consistent, and high-quality dataset that is suitable for subsequent analytical processing. By performing normalization, missing value handling, duplicate removal, record validation, and feature generation, the resulting dataset becomes more reliable, reproducible, and ready for further statistical and exploratory analysis.

4.2 DATA STRUCTURE

The cleaned dataset produced by the data cleaning pipeline is stored as nst_cleaned.csv and is organized in a structured tabular format, where each row represents a unique news article collected from the New Straits Times (NST) website, and each column represents a specific article attribute. The dataset comprises the original metadata extracted during the web crawling process together with several derived analytical attributes generated during data cleaning, providing both descriptive and quantitative information about each article. This standardized schema facilitates efficient data storage, retrieval, and subsequent analytical processing while supporting reproducible analysis and future data-driven applications. **Table 4.1** presents the complete dataset schema, including the data type, description, and representative example for each attribute.

Table 4.1: Schema of the Cleaned NST Dataset

Column	Data Type	Description
record_id	String	Unique identifier assigned to each article record. In this implementation, the article URL is used as the record identifier.
url	String	Full URL of the news article, used as the primary unique key for record identification and deduplication.
headline	String	Title of the news article was extracted from the webpage.
publication_date	DateTime (ISO 8601)	Date and time when the article was originally published, extracted from the article's metadata.
modified_date	DateTime (ISO 8601)	Date and time when the article was last modified, if available.
section	String	News category or section to which the article belongs.
author	String	Name of the article author extracted from the metadata.
summary	String	Short summary or description of the article extracted from the metadata.
keywords	String	Comma-separated keywords or tags associated with the article.

body_preview	String	Preview text generated from the first paragraphs of the article body.
headline_length	Integer	Total number of characters in the normalized headline.
summary_length	Integer	Total number of characters in the normalized summary.
word_count	Integer	Total number of words contained in the headline, summary, and body preview.
collected_at	DateTime	Timestamp indicating when the article was collected by the crawler.

The standardized dataset schema preserves both original article metadata and the derived analytical attributes, providing a comprehensive representation of each news article. Furthermore, storing the processed dataset in CSV format enhances interoperability with data analysis libraries such as Pandas and Polars, spreadsheet applications, and statistical software, thereby improving the portability, accessibility, and usability of the dataset for subsequent analysis and reporting.

4.3 TRANSFORMATION & FORMATTING

Following the data cleaning stage, data transformation and formatting were performed to prepare the cleaned dataset for efficient analysis and reporting. The transformation process was divided into two stages. The first stage was implemented in the clean_nst.py module, which performs the core dataset cleaning, formatting, feature generation, and exports the cleaned dataset as data/processed/nst_cleaned.csv. The second stage was implemented in the polars_nst.py module, which utilizes the Polars LazyFrame execution model to perform analytical transformations and generate summary datasets for exploratory analysis.

4.3.1 TEXT NORMALIZATION

The textual attributes, including headline, summary, and body_preview, were normalized by removing unnecessary whitespace characters and standardizing text formatting. This process ensures consistency across all textual fields, improving data quality, and facilitating reliable text-based analysis.

4.3.2 DATA TYPE ENFORCEMENT

During the cleaning process, the dataset attributes were stored using appropriate data types to improve consistency and processing efficiency. Textual metadata such as the headline, author, and section were maintained as string values, whereas the derived analytical attributes (headline_length, summary_length, and word_count) were stored as integer values. This standardization enables accurate filtering, aggregation, and statistical analysis.

4.3.3 DEDUPLICATION

A deduplication process was applied using the url field as the unique identifier to ensure that each news article appeared only once in the cleaned dataset. Removing duplicate records improves data integrity and prevents repeated articles from influencing subsequent analyses.

4.3.4 DATA SERIALIZATION

After the cleaning and formatting processes were completed, the processed dataset was exported as nst_cleaned.csv using UTF-8 encoding with a Byte Order Mark (UTF-8 BOM). This encoding preserves multilingual characters and ensures compatibility with spreadsheet applications such as Microsoft Excel as well as various statistical and data analysis tools.

4.3.5 ANALYTICAL TRANSFORMATION USING POLARS

Following the generation of the cleaned dataset, additional analytical transformations were performed using the polars_nst.py module. The module leverages the Polars LazyFrame execution model to efficiently process the cleaned data and generate summary statistics without modifying the original dataset.

The analytical processing includes:

- Casting and normalizing string columns to ensure consistent text processing.
- Removing duplicate records based on the url field prior to analysis.
- Grouping articles by section to produce category-level article distributions.

- Grouping articles by publication year and month to generate monthly publication summaries.
- Ranking authors according to the number of published articles to identify the most active contributors.

The analytical outputs are exported as separate CSV files in the data/processed/polars_outputs/ directory, including:

- category_summary.csv – summarizes the number of articles published in each news category.
- monthly_summary.csv – summarize article publication trends per month.
- top_authors.csv – ranks authors according to the number of published articles.

By separating the responsibilities of clean_nst.py and polars_nst.py, the proposed pipeline adopts a modular architecture in which data cleaning and analytical processing are performed independently. This design improves the maintainability, scalability, and reusability of the data processing workflow while allowing additional analytical components to be incorporated in future developments without affecting the core cleaning pipeline.

5.0 OPTIMIZATION TECHNIQUES

This section describes the optimization techniques implemented to improve the performance of the proposed NST web crawling and data processing pipeline. Three optimization methods were applied throughout the project. The first optimization focuses on improving the web crawling stage using multithreading to download multiple web pages concurrently. The second optimization accelerates data cleaning by utilizing multiprocessing to distribute computational tasks across multiple CPU cores. Finally, Polars Lazy Execution is employed to optimize analytical data processing through query optimization and parallel execution. The effectiveness of these techniques is evaluated in the subsequent performance evaluation section using execution time, CPU usage, memory usage, and throughput.

5.1 METHOD 1 – MULTITHREAED WEB CRAWLING

The first optimization technique implemented in this project is multithreaded web crawling using Python's **ThreadPoolExecutor**. The baseline crawler downloads news articles sequentially, where each webpage is requested and processed one at a time. Although this approach is simple to implement, it spends a considerable amount of time waiting for network responses, resulting in low crawling efficiency when collecting a large number of articles.

To improve the crawling performance, the proposed crawler utilizes ThreadPoolExecutor to execute multiple download tasks concurrently. Instead of waiting for one webpage to finish before requesting the next, multiple worker threads retrieve different article pages simultaneously. Since web crawling is an I/O-bound task, multithreading allows the program to make better use of idle waiting time and significantly reduces the overall execution time.

Each thread independently downloads an article, extracts the required metadata using BeautifulSoup, and returns the processed result to the main program. The extracted records are then combined and stored in the same output format as the baseline crawler, ensuring both implementations produce identical datasets for a fair performance comparison. This optimization substantially increases the crawler throughput while maintaining the accuracy and consistency of the collected data.

```python
def crawl_threaded(urls: list[str], args: argparse.Namespace, robots: RobotFileParser, visited: set[str]) -> int:
    total = 0
    pending = [url for url in urls if url not in visited]
    with ThreadPoolExecutor(max_workers=args.workers) as executor:
        futures = {executor.submit(crawl_one, url, args.user_agent, robots, args.delay): url for url in pending}
        for future in as_completed(futures):
            if total >= args.target_records:
                break
            url = futures[future]
            record = future.result()
            if record:
                append_jsonl([record])
                append_visited([url])
                total += 1
                log(f"[threaded] {total:,}/{args.target_records:,} {record['headline'][:80]}")
    return total
```

Figure 5.1.1: Implementation of Multithreaded Web Crawling using Python ThreadPoolExecutor.

5.2 METHOD 2 – PARALLEL DATA CLEANING

The second optimization technique focuses on improving the efficiency of data cleaning through Python's **multiprocessing** module. After the crawling process is completed, the collected dataset undergoes several preprocessing operations, including whitespace normalization, duplicate removal, missing value handling, record validation, and feature generation. When these operations are executed sequentially on a large dataset, the processing time increases considerably because only a single CPU core is utilized.

To overcome this limitation, multiprocessing was implemented by dividing the cleaned dataset into several smaller partitions. Each partition is assigned to a separate worker process, allowing multiple CPU cores to execute the same cleaning operations simultaneously. Unlike multithreading, multiprocessing creates independent Python processes, enabling true parallel computation for CPU-intensive workloads.

After all worker processes complete their assigned tasks, the cleaned partitions are merged into a single dataset before being exported as the final processed CSV file. By distributing the workload across multiple CPU cores, the multiprocessing approach reduces the overall data cleaning time while preserving data consistency and correctness.

```
started = time.perf_counter()

# Step 11: Compare basic sequential cleaning with multiprocessing cleaning.
if args.mode == "basic":
    cleaned = [clean_record(record) for record in records]
else:
    with Pool(processes=args.workers) as pool:
        cleaned = pool.map(clean_record, records)
seconds = time.perf_counter() - started
```

Figure 5.2.1: Multiprocessing Implementation for Parallel Data Cleaning using Python Pool.

5.3 METHOD 3 – POLARS LAZY EXECUTION

The third optimization technique implemented in this project is **Polars Lazy Execution**, which improves the efficiency of analytical data processing. Although Pandas provides a convenient interface for manipulating structured data, it follows an eager execution model where every

transformation is executed immediately. As the dataset grows larger, this behaviour introduces unnecessary intermediate computations and increases both execution time and memory consumption.

To address these limitations, Polars LazyFrame was adopted to perform analytical processing. Instead of executing each operation immediately, Polars constructs an optimized execution plan by recording every transformation requested by the program. The operations are executed only when the **collect()** function is called, allowing the query optimizer to combine multiple transformations, eliminate redundant computations, and execute the workload efficiently.

In addition to query optimization, Polars automatically performs parallel execution across multiple CPU cores, further improving processing performance. This approach minimizes unnecessary memory allocation while significantly reducing execution time compared with the baseline Pandas implementation. As a result, Polars Lazy Execution provides the highest processing efficiency among the implemented optimization techniques and is well suited for handling large-scale analytical datasets.

```
lf = (
    pl.scan_csv(str(input_file), infer_schema_length=10000)
    .select(
        [
            pl.col("url").cast(pl.Utf8),
            pl.col("headline").cast(pl.Utf8),
```

```
# Analytical outputs for report.
category_summary = (
    lf.group_by("section_clean")
    .agg(
        [
            pl.len().alias("article_count"),
            pl.col("headline_length_polars").mean().alias("avg_headline_length"),
            pl.col("summary_length_polars").mean().alias("avg_summary_length"),
            pl.col("word_count").mean().alias("avg_word_count"),
        ]
    )
    .sort("article_count", descending=True)
    .collect()
)
```

Figure 5.3.1: Implementation of Polars LazyFrame showing Lazy Query Construction and Execution using collect().

6.0 PERFORMANCE EVALUATIONS

This section evaluates the effectiveness of the optimization techniques implemented in the proposed NST web crawling and data processing pipeline. Performance benchmarking was conducted by comparing the baseline implementations with the optimized versions using the same workload. The evaluation focuses on four performance metrics, namely execution time, CPU usage, memory usage, and throughput (records processed per second). These metrics provide quantitative evidence of the improvements achieved through multithreading, multiprocessing, and Polars Lazy Execution.

6.1 BEFORE VS AFTER OPTIMIZATION

The main objective of this performance evaluation is to quantitatively assess the effectiveness of the optimization techniques implemented in the proposed NST web crawling and data processing pipeline. The evaluation compares the baseline implementations with the optimized implementations using the same workload to ensure a fair and consistent comparison. Performance benchmarking was conducted on both the web crawling and data processing stages using the collected dataset containing **102,265** news article metadata records. The benchmark process is divided into two stages, namely **before optimization** and **after optimization**.

Before Optimization

Before optimization, the system was evaluated using the baseline implementations. The web crawling stage employed a sequential crawler that downloaded and processed one article at a time, while the data processing stage used the standard Pandas implementation to perform data cleaning and transformation sequentially. These baseline implementations served as the reference point for evaluating the effectiveness of the proposed optimization techniques.

After Optimization

After optimization, the system incorporated three optimization techniques to improve overall performance. During the web crawling stage, **ThreadPoolExecutor** was implemented to

download multiple article pages concurrently, reducing idle waiting time caused by network communication. During the data processing stage, **multiprocessing** was applied to distribute cleaning tasks across multiple CPU cores, enabling parallel execution of computationally intensive operations. In addition, **Polars Lazy Execution** was adopted to optimize analytical processing through lazy evaluation, query optimization, and automatic parallel execution. The effectiveness of these optimization techniques is evaluated in terms of execution time, CPU utilization, memory consumption, and throughput, which are presented in the following section.

6.2 COMPARISON OF TIME, MEMORY, CPU USAGE, THROUGHPUT

This section presents the benchmark results obtained from the performance evaluation of the proposed NST web crawling and data processing pipeline. The evaluation compares the baseline implementations with the optimized implementations using four performance metrics, namely execution time, CPU utilization, memory usage, and throughput. The benchmark results are presented separately for the web crawling stage and the data processing stage to provide a comprehensive assessment of the implemented optimization techniques.

6.2.1 WEB CRAWLING PERFORMANCE COMPARISON

Table 6.1 summarizes the benchmark results obtained from the baseline crawler and the multithreaded crawler using the same workload of 500 news articles.

Table 6.1: Performance Comparison of Web Crawling

Crawler	Records	Execution Time (s)	Throughput (records/s)	CPU Usage (%)	Memory Usage (MB)
Baseline	500	508.876	0.983	13.86	66.87
Threaded	500	165.303	3.025	30.01	69.95

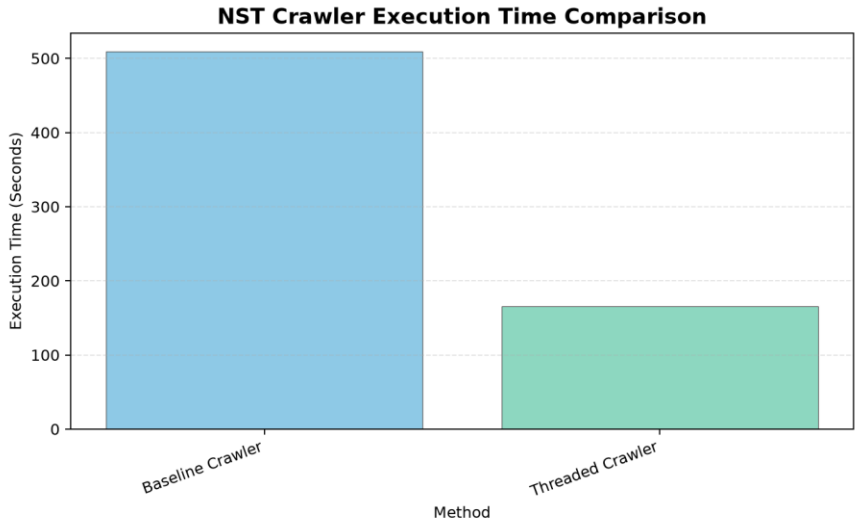

Figure 6.1: Execution Time Comparison for Web Crawling

Figure 6.1 shows that the execution time comparison demonstrates that the multithreaded crawler significantly outperformed the baseline crawler. By downloading multiple articles concurrently, the threaded implementation reduced the crawling time from **508.876 seconds** to **165.303 seconds**, representing an improvement of approximately **67.5%**. The reduction in execution time confirms that multithreading effectively minimizes idle waiting time during network communication.

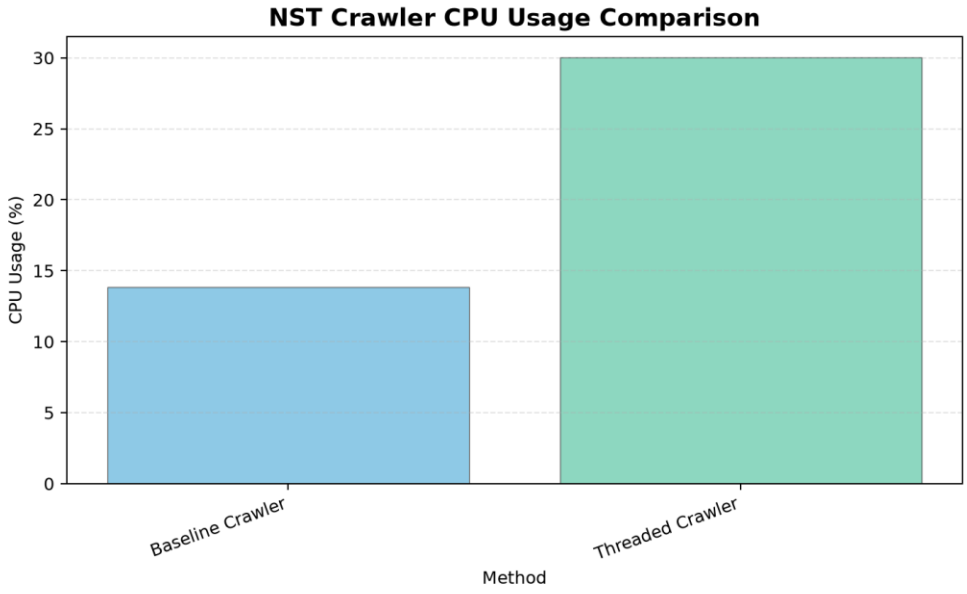

Figure 6.2: CPU Usage Comparison for Web Crawling

The CPU usage increased from **13.86%** to **30.01%** after applying multithreading, as shown in Figure 6.2. This increase is expected because multiple worker threads execute simultaneously to maximize processor utilization. Although CPU utilization increased, the higher resource usage contributed directly to faster crawling performance.

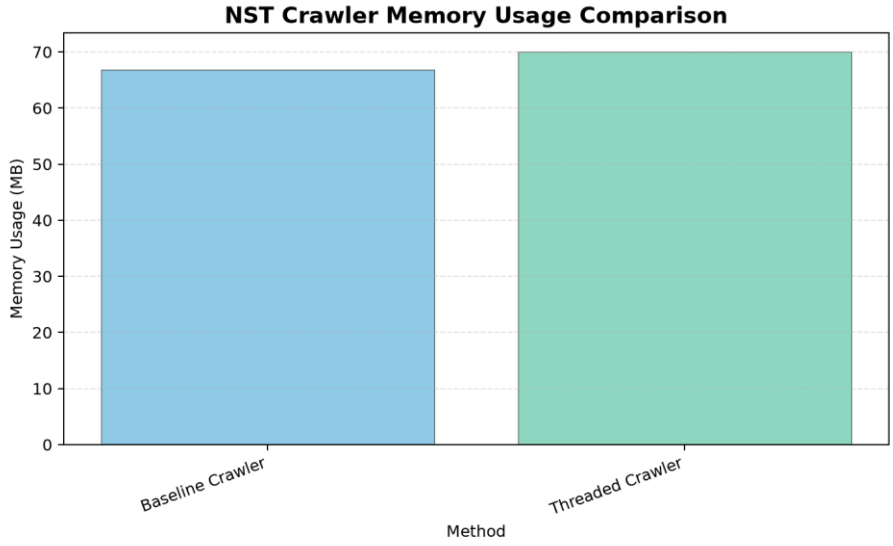

Figure 6.3: Memory Usage Comparison for Web Crawling

Figure 6.3 shows that the memory consumption increased only slightly from **66.87 MB** to **69.95 MB**. The small increase indicates that the multithreaded implementation introduces minimal memory overhead while providing substantial performance improvements.

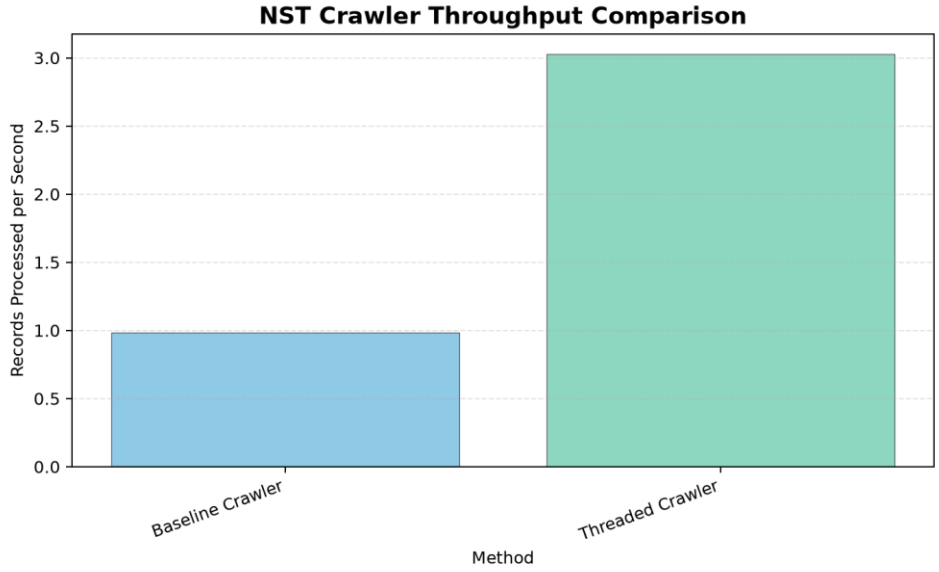

Figure 6.4: Throughput Comparison for Web Crawling

Figure 6.4 shows that the throughput increased from **0.983** to **3.025 records per second**, allowing the crawler to process approximately three times more articles within the same period. This demonstrates that multithreading substantially improves the overall efficiency and scalability of the crawling process.

6.2.2 DATA PROCESSING PERFORMANCE COMPARISON

Table 6.2 presents the benchmark results obtained from the three data processing implementations, namely baseline Pandas, multiprocessing, and Polars Lazy Execution.

Table 6.2: Performance Comparison of Data Processing

Method	Records	Execution Time (s)	Throughput (records/s)	CPU Usage (%)	Memory Usage (MB)
Pandas	102,265	7.510	13617.895	92.0	466.40
Multiprocessing	102,265	6.231	16413.567	75.0	528.96
Polars Lazy Execution	102,265	0.558	183392.279	29.6	332.92

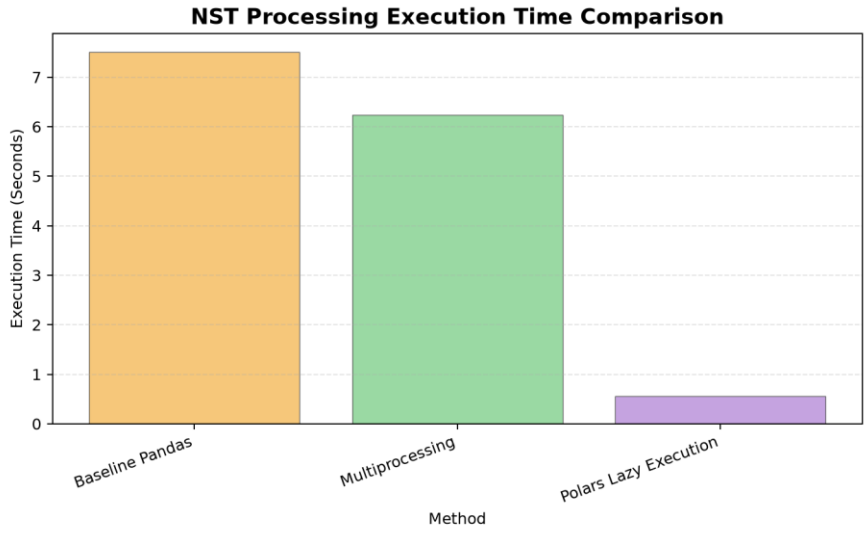

Figure 6.5: Execution Time Comparison for Data Processing)

As shown in Figure 6.5, among the three implementations, **Polars Lazy Execution** achieved the shortest execution time of **0.558 seconds**, significantly outperforming both the baseline Pandas (**7.510 seconds**) and multiprocessing (**6.231 seconds**) implementations. The result demonstrates

the effectiveness of Polars' lazy evaluation and query optimization in reducing unnecessary computations.

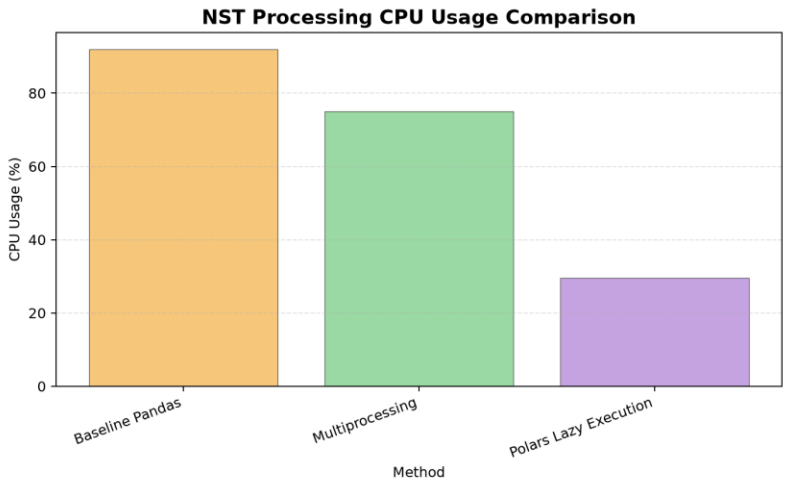

Figure 6.6: CPU Usage Comparison for Data Processing

Figure 6.6 shows that the baseline Pandas implementation recorded the highest CPU utilization (**92.0%**), followed by multiprocessing (**75.0%**), while Polars Lazy Execution required only **29.6%** CPU usage. The lower CPU utilization achieved by Polars indicates more efficient execution despite delivering the fastest processing performance.

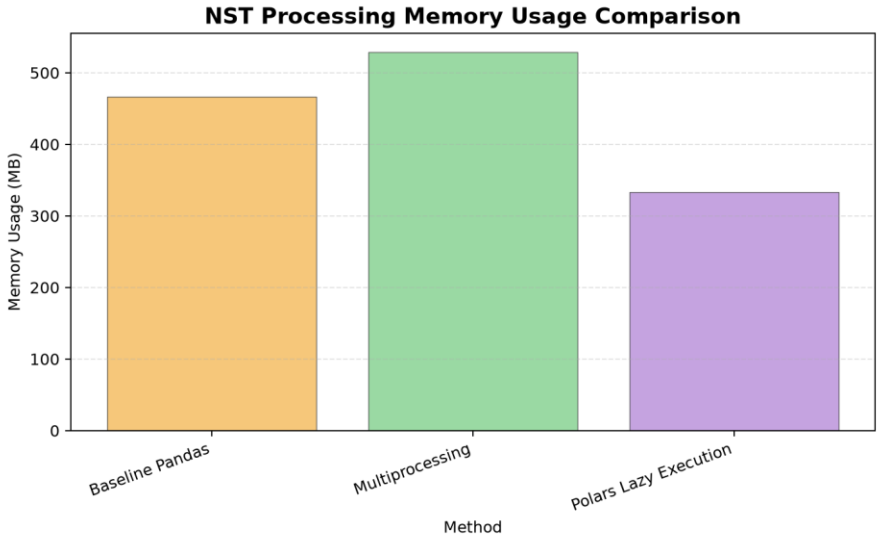

Figure 6.7: Memory Usage Comparison for Data Processing

Figure 6.7 illustrates that multiprocessing recorded the highest memory consumption (**528.96 MB**) because multiple worker processes were created during execution. In contrast, Polars Lazy Execution required only **332.92 MB**, making it the most memory-efficient implementation among the evaluated methods.

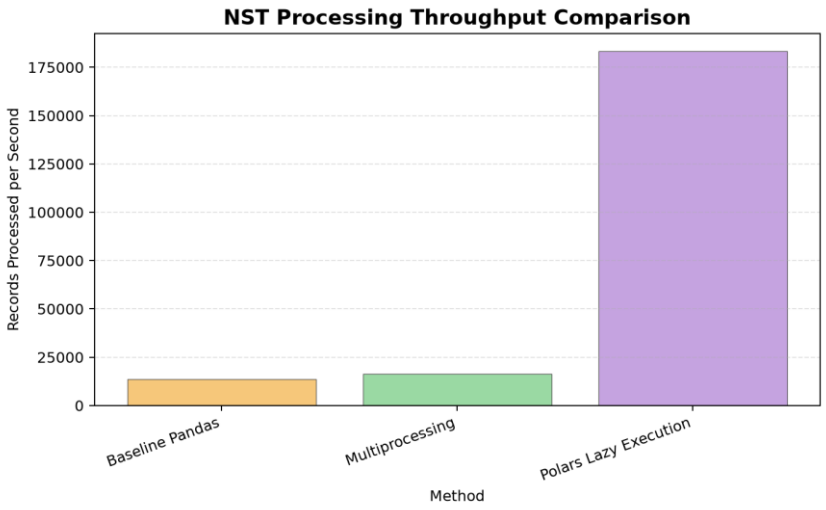

Figure 6.8: Throughput Comparison for Data Processing

Throughput improved considerably after optimization. The baseline Pandas implementation processed **13,617.895 records per second**, while multiprocessing increased the throughput to **16,413.567 records per second**. Polars Lazy Execution achieved the highest throughput of **183,392.279 records per second**, demonstrating its superior capability for processing large-scale datasets efficiently.

Overall, the benchmark results indicate that the implemented optimization techniques significantly improve the performance of both the web crawling and data processing stages. Multithreading provides substantial improvements in crawling efficiency by reducing execution time and increasing throughput, while Polars Lazy Execution delivers the best overall performance for analytical data processing by combining the shortest execution time, lowest CPU utilization, lowest memory consumption, and highest throughput.

7.0 CHALLENGES & LIMITATIONS

This section discusses the challenges encountered during the implementation of the proposed NST web crawling and data processing pipeline. It also highlights the limitations of the implemented optimization techniques and identifies factors that may influence the overall system performance.

7.1 CHALLENGES ENCOUNTERED

Several challenges were encountered during the development and benchmarking of the proposed system. One of the primary challenges was the dependency of the web crawler on network conditions. Since the crawler retrieves article pages directly from the New Straits Times (NST) website, fluctuations in internet connectivity, server response time, and network latency could affect the overall crawling performance. Consequently, the execution time recorded during benchmarking may vary slightly across different runs even when the same workload is used.

Another challenge involved balancing crawling performance with responsible web crawling practices. Although increasing the number of worker threads could further improve crawling speed, excessive concurrent requests may place unnecessary load on the target website. Therefore, the number of worker threads and request delays were carefully configured to achieve a balance between performance improvement and responsible data collection.

In addition, implementing multiprocessing for data cleaning introduced synchronization and process management overhead. While multiple CPU cores reduced the overall execution time, additional system resources were required to create and manage independent worker processes before merging the processed results into a single dataset.

7.2 LIMITATIONS

Although the proposed optimization techniques significantly improved system performance, several limitations remain. The benchmark results presented in this project were obtained using the NST news dataset and may differ when applied to websites with different structures, larger

multimedia content, or more dynamic webpages. Consequently, the observed performance improvements should not be generalized to all web crawling applications.

Furthermore, the effectiveness of Polars Lazy Execution depends on the characteristics of the analytical workload. Although Polars achieved the best performance for the implemented data processing tasks, different datasets or more complex analytical operations may produce different execution characteristics.

Finally, the performance evaluation was conducted on a specific hardware and software environment. Variations in processor specifications, available memory, operating systems, and network conditions may influence execution time, CPU utilization, memory consumption, and throughput. Therefore, the benchmark results should be interpreted within the context of the experimental environment used in this project.

Despite these limitations, the proposed system successfully achieved the project objectives by collecting more than **102,000** news article metadata records and demonstrating significant performance improvements through multithreading, multiprocessing, and Polars Lazy Execution. The implemented optimization techniques effectively improved both web crawling and data processing performance while maintaining the quality and consistency of the collected dataset.

8.0 CONCLUSION & FUTURE WORK

This section summarizes the achievements of the proposed NST web crawling and data processing pipeline and presents several recommendations for future improvements that could further enhance the system's performance, scalability, and functionality.

8.1 SUMMARY OF FINDINGS

This project successfully developed a high-performance web crawling and data processing pipeline for collecting large-scale news article metadata from the New Straits Times (NST) website. By utilizing the NST XML sitemap, the developed crawler efficiently discovered article URLs and automatically extracted structured metadata, including the article headline, publication date, author, section, summary, keywords, and body preview. The crawler successfully collected **102,265** news article metadata records, exceeding the project target of **100,000** records.

To improve system performance, three optimization techniques were implemented throughout the project. ThreadPoolExecutor was adopted to perform multithreaded web crawling, enabling multiple article pages to be downloaded concurrently and significantly reducing the overall crawling time. Multiprocessing was applied to parallelize the data cleaning stage by distributing cleaning tasks across multiple CPU cores. In addition, Polars Lazy Execution was employed to optimize analytical data processing through lazy evaluation, query optimization, and automatic parallel execution.

The performance evaluation demonstrated that the implemented optimization techniques substantially improved the efficiency of both the crawling and data processing stages. The multithreaded crawler achieved a significant reduction in execution time while increasing throughput compared with the baseline crawler. Similarly, Polars Lazy Execution achieved the best overall performance among the evaluated processing methods by recording the shortest execution time, the highest throughput, and lower CPU and memory utilization than the baseline implementation. These findings demonstrate that the proposed optimization techniques effectively enhance the scalability and efficiency of large-scale web crawling and data processing tasks.

Overall, the project successfully achieved all the stated objectives by developing an automated web crawler, collecting a large-scale structured dataset, implementing multiple optimization techniques, performing comprehensive data cleaning and transformation, and evaluating the effectiveness of the proposed approaches through performance benchmarking.

8.2 FUTURE IMPROVEMENT

Although the proposed system achieved satisfactory performance, several improvements can be considered in future work to further enhance its capabilities. First, additional asynchronous crawling techniques, such as AsyncIO or asynchronous HTTP libraries, may be incorporated to further improve crawling performance and better utilize network resources when collecting a larger number of webpages.

Second, the crawler may be extended to support multiple Malaysian news websites instead of focusing solely on the New Straits Times (NST). Collecting data from multiple news sources

would increase dataset diversity and provide broader coverage for subsequent data analysis and research applications.

Furthermore, the processed dataset could be integrated with distributed big data processing frameworks such as Apache Spark or Dask to support large-scale data processing across multiple computing nodes. Such an enhancement would improve scalability when handling datasets containing millions of records.

Finally, additional analytical capabilities may be incorporated into the processing pipeline, including topic classification, sentiment analysis, keyword extraction, and news trend analysis using machine learning or natural language processing techniques. These enhancements would increase the analytical value of the collected dataset and enable more advanced applications in news analytics and decision support.

In conclusion, the proposed NST web crawling and data processing pipeline provides an effective and scalable solution for large-scale metadata collection and analysis. The implemented optimization techniques successfully improved system performance while maintaining data quality, demonstrating the benefits of high-performance computing approaches in modern web crawling applications.

9.0 REFERENCES

New Straits Times (NST Online) | Malaysia News & World Updates. (n.d.). NST Online.

https://www.nst.com.my/

10.0 APPENDICES